演習01：マップとステージ管理

担当案：代表者 kusupon / 共通準備と統合も担当

到達目標

固定の学校を2種類のステージへ拡張し、壁・扉・経路・視界をWorldで一元管理します。各担当が同じマップを参照できる状態が完成条件です。

先に00_commonの4〜9節を読み、starter/contracts.pyをゲームのルートに置きます。既存のmake_map、center、cell_at、walkable、find_path、visible、WALLSが主な移動対象です。

必須：2ステージ、World、到達可能性の確認、タイトルでの選択。任意：3ステージ目、装飾、ランダム生成。

練習1：マップをデータにする

stages.pyを作り、現在のGRIDを最初のStageへ移します。最初は表示や挙動を変えません。第2ステージは第1ステージを深くコピーしてから通路を変え、共有リストを直接変更しないようにします。

Stageの項目はcontracts.pyの定義どおりです。gridは0が床、1が固定壁。鍵は {鍵ID: Cell}、扉・スイッチ・隠れ場所は独立したデータです。

```
stage = Stage(
    stage_id='school_a', name='校舎A', grid=grid_a,
    player_spawn=(4, 3), guard_routes=routes_a,
    keys={'red': (22, 3), 'blue': (4, 14), 'green': (22, 14)},
    exit_cell=(13, 15), heal_cell=(24, 13),
    hiding_spots=((6, 3),), doors=(), switches=(),
    required_keys=frozenset({'red', 'blue', 'green'}),
)
```

最初は扉とスイッチを空にします。これで従来のゲームが動くことを確認してから仕掛け担当のデータを追加します。

確認：2つのStageのgridを書き換えても、もう片方のgridは変わらない。

練習2：座標と通行判定を移す

World(stage)を作り、stageとopened_doorsを保持します。centerとcell_atは現在の処理をメソッドへ移し、TILEとORIGINはsettings.pyから読みます。

walkableは「範囲内」「固定壁でない」「未開放の扉でない」の3条件を確認します。負の添字で反対側のマスを参照しないよう、範囲判定を先に行います。

```
def walkable(self, cell: Cell) -> bool:
    col, row = cell
    # TODO: 範囲外ならFalse
    # TODO: 固定壁ならFalse
    # TODO: このマスの扉が未開放ならFalse
    return True
```

wallsには固定壁と閉じた扉のRectを含めます。Player.moveの for wall in WALLS を for wall in world.walls に変更し、updateからmoveへworldを渡します。

確認：範囲外はFalse、床はTrue、壁はFalse。位置とRectの対応が1マスずれていない。

練習3：経路と視線を移す

既存find_pathのwalkable呼び出しをself.walkableに置き換えます。経路は開始マスを除き、目的地を含めます。経路を作れない場合は空リストを返します。

visibleもself.walkableで判定します。扉の前後で視線と通行の結果が一致するようにします。直接GRIDだけを見ている処理が残っていないか検索します。

open_doorはdoor_idが未開放ならopened_doorsへ追加し、wallsを再構築してTrueを返します。既に開いているときはFalse、存在しないIDにはValueErrorを使う、という仕様で統一します。

確認：閉じた扉で経路が遮られ、開いた後は同じstart/goalで経路が得られる。敵担当へ変更の反映タイミングを伝える。

練習4：配置を検証する

validate_stage(stage)を作り、問題があれば理由付きのValueErrorを返します。問題なければ何も返さず終了します。

1. gridの行数・列数が揃い、値が0/1であるか。

2. 開始、出口、鍵、回復、隠れ場所、巡回地点、扉、スイッチが範囲内の床か。

3. IDの重複、存在しない鍵を要求する扉、存在しない扉を指すスイッチがないか。

4. 全ての巡回地点に、扉を全開にした床マップ上で開始地点から到達できるか。

5. 鍵・スイッチを順番に使えば、必須鍵と出口まで進めるか。

5は全扉を閉じて到達できるかだけでは判定できません。初期状態で行ける範囲を探索し、届く鍵を拾い、届くスイッチと取得鍵で扉を開きます。新しい鍵・扉がなくなるまで繰り返し、最後に必須鍵と出口を検査します。

この検査は「仕掛けとして進行可能か」を調べるものです。敵を避けられるか、時間内に終わるかは実際に遊んで別に確認します。

練習5：タイトルから選ぶ

Game.__init__でstage_index=0を設定します。タイトル中に1/2キーで選び、Enterを押すと選んだStageでresetします。プレイ中にステージ変更はしません。

```
# Game.reset内の接続例
self.world = World(STAGES[self.stage_index])
self.player = Player(self.world.stage.player_spawn, self.world)
self.guards = [Guard(route, self.world)
               for route in self.world.stage.guard_routes]
```

これに合わせてPlayer/Guardのコンストラクタを変更します。旧center呼び出しはworld.centerへ置き換えます。各クラスがgame.pyをimportしない構成にしてください。

再挑戦は同じステージを初期状態に戻します。結果画面でTを押すとstate='title'へ戻る分岐をGame.handle_eventへ追加します。タイトルでは1/2で選び直せます。プレイ中のTは無効です。タイトルと結果画面の操作案内にもTを追記します。

練習6：描画と統合

World.draw(screen)でgridと扉を描きます。教室名と机の位置はStageに追加するか、まず装飾を省略して衝突判定と一致させます。旧Game.draw_mapの固定配置をそのまま第2ステージへ描かないでください。

鍵やスイッチはGimmicks.draw、キャラクターは各クラスが描きます。Gameは描画順だけを管理します。world.open_doorを呼んだフレームの後に敵を更新します。

代表者は各機能を1つずつ接続し、起動と基本操作を確認してから次を取り込みます。

動作確認表

入力・準備	期待する結果
タイトルで1 / 2を選ぶ	それぞれ違うマップになる
不正な鍵座標を設定	起動前の検証で位置が分かるエラー
自分の鍵を扉の奥へ置く	進行不能として検出
扉を2回開く	初回True、2回目False
壁越しに敵と主人公を置く	visibleがFalse
再挑戦	開けた扉、取得鍵、敵位置が初期状態

提出前の確認

world.py、stages.py、必要な共通接続、READMEの担当欄をコミットします。基本機能の移動と新しい2ステージ・配置検証を区別して説明します。

PRには、Stage追加方法、World API、他担当が直す呼び出し箇所、動作確認の結果を記載します。

基準コード：https://github.com/kusupon/ProjExD_Group13/blob/33fe9905c183b321d1988d66041feb8b7e74998f/game.py